CivicRecords AI — User Manual

Version 1.1+ · April 2026

> **Release recovery notice (2026-05-11).** CivicRecords AI v1.6.0 is the Docker secret-file extraction release that closes QA-002 on top of the v1.5.0 CivicCore v1.0.1 recovery alignment. The older `v1.4.10` tag remains available as historical pre-gate, provisional source only and must not be promoted as an attested baseline.

> This manual serves three audiences. Jump to your section: > - **Section A — End-User Guide** — City clerks, records officers, and staff who process requests daily. No technical background required. > - **Section B — Technical Reference** — IT administrators, system integrators, and power users who configure and maintain the system. > - **Section C — Architectural Reference** — Developers and architects who need to understand how the system is built and why.

Section A — End-User Guide

Written for staff who receive, process, and fulfill open records requests. You do not need to understand the technology to use this manual.

A.1 What Is CivicRecords AI?

CivicRecords AI is a tool your city uses to respond to public records requests — the formal requests that residents, journalists, and attorneys submit asking for government documents.

Before this tool, staff had to search through file shares, email archives, and multiple databases by hand, then review every document one by one to check for sensitive information. That process could take hours or days for a single request.

CivicRecords AI automates the searching, surfaces the most relevant documents, flags information that may be legally protected, and tracks every request from start to finish.

What you can do with it:

- Search across all your city's connected documents using plain English questions
- Create and manage records requests from intake through fulfillment
- Review documents flagged for potentially sensitive information (you always make the final call — the system never releases or redacts anything automatically)
- Generate AI-assisted response letters
- Track deadlines and request status in real time

What it does not do:

- Release documents without human approval
- Redact information automatically
- Replace your legal judgment — it assists it

A.2 Signing In

1. Open your browser and go to the address your IT department provided (typically `http://[your-city-server]:8080`). 2. Enter your email address and password. 3. Click **Sign In**.

If you cannot sign in, contact your system administrator. Passwords are not stored in this system — your administrator must reset your account.

Your role determines what you see:

- **Admin** — Full access: all departments, sources, users, and settings
- **Staff** — Your department's requests and documents only
- **Liaison** — Read-only view of your department's requests

If you see less than you expect, your role or department assignment may need to be updated. Contact your administrator.

A.3 The Dashboard

After signing in, you see the main dashboard. On a desktop or tablet browser, the navigation lives in the left sidebar. On a phone or narrow browser window (under ~768 pixels wide), the sidebar collapses behind a menu button (three horizontal lines, ☰) in the top-left corner — tap it to slide the menu in, tap a section to navigate (the menu closes by itself), or tap the ✕ inside the menu to close it without navigating.

The menu has these sections:

| Section | What it does | |---|---| | **Search** | Search all connected city documents | | **Requests** | Open records request inbox and lifecycle | | **Sources** | Connected data systems (Admin only) | | **Exemptions** | Flagged content for human review | | **Admin** | User accounts, departments, settings |

The top of the screen shows your name, role, and a notification bell for pending items.

Services row (admins). Scroll down the dashboard to find the **SERVICES** card. It shows one indicator per backing service: **Database (PostgreSQL)**, **Ollama (LLM Engine)**, and **Redis (Task Queue)**. A green check (✓) means the service is reachable and healthy; a red X (✕) means the dashboard couldn't reach it. If you see a red X but the rest of the app seems to work, that service may have just restarted — refresh the page. If the red X persists, see Section D.1 (Troubleshooting).

A.4 Searching Documents

Search is the fastest way to find documents responsive to a records request.

To search: 1. Click **Search** in the left sidebar. 2. Type your question in plain English. For example: - *"contracts with Apex Roofing signed after 2022"* - *"incident reports from the Riverside District in March 2024"* - *"employee termination records for public works department"* 3. Press **Enter** or click **Search**.

Reading results:

- Each result shows the document title, the source system it came from, a relevance score, and a text excerpt.
- Click a result to open it.
- The **AI Summary** toggle (if enabled) shows a one-paragraph summary of the document — useful for quickly deciding if it's responsive before opening the full file.

Filters:

- Use the date range filter to narrow results by when documents were created or modified.
- Use the source filter to search only specific connected systems.

Export:

- Click **Export CSV** to download the result list. The CSV includes document titles, sources, relevance scores, and source paths — useful for documenting your search process in the request record.

A.5 Managing Records Requests

Creating a New Request

1. Click **Requests** → **New Request**. 2. Fill in the form: - **Requester name and contact information** - **Request description** — what the requester is asking for - **Date received** - **Legal deadline** — your jurisdiction's response deadline (e.g., 3 business days for CORA, 5 business days for most state FOIA equivalents) 3. Click **Create Request**.

The request is assigned a reference number and enters the **Intake** status.

The Request Lifecycle

Requests move through ten statuses. You advance a request manually — the system never skips a status automatically.

| Status | Meaning | |---|---| | **Intake** | Request received and logged | | **Clarification** | Waiting for additional information from the requester | | **Assigned** | Assigned to a staff member for processing | | **Search** | Staff is searching for responsive documents | | **Review** | Documents found; under legal review | | **Drafting** | Response letter being prepared | | **Approval** | Waiting for supervisor or legal approval | | **Fulfillment** | Documents and letter sent to requester | | **Closure** | Request complete and closed |

To advance a request: open it → click **Advance Status** → select the next status → confirm.

Searching Within a Request

From inside a request, click **Run Search** to search for responsive documents. The system uses the request description to generate an initial query, which you can edit. Results are saved to the request record for audit purposes.

Adding Documents to a Request

After searching, mark documents as **Responsive** or **Non-Responsive**. Only documents marked responsive will appear in the response package.

Request Timeline

Every action on a request is recorded in the **Timeline** tab: who did what, and when. This is your audit trail.

A.6 Exemption Detection and Review

Exemption detection finds content that may be legally protected from release — personal identification numbers, medical information, law enforcement-specific data, and state statutory exemptions.

Important: The system flags content for your review. It does not redact or withhold anything. Every flag requires your decision.

How Flags Work

When the system analyzes a document, it may produce flags:

- **Tier 1 — Pattern matches:** Social Security numbers, credit card numbers, phone numbers, email addresses, bank account numbers, and driver's license formats (all 50 states and DC).
- **Tier 2 — Statutory keywords:** Phrases matching exemption language in your state's open records law — **175 state-scoped rules across 51 jurisdictions** (50 states + DC), auto-seeded on first boot by T5B. The seed source additionally defines 5 universal PII rules (180 total in `STATE_RULES_REGISTRY`) that are **not** currently seeded automatically because they lack a two-letter `state_code`; those are deferred pending a schema relaxation.
- **Tier 3 — LLM review (optional):** If enabled, an AI model reviews flagged content and adds context.

Reviewing a Flag

1. Open the request → click **Exemption Review**. 2. Each flag shows: - The flagged text excerpt - The exemption rule that triggered it - The page or document location 3. Choose: - **Confirm** — This information is exempt; do not include it in the release. - **Dismiss** — This is not actually sensitive; include it in the release. - **Escalate** — Flag for legal review before deciding.

All decisions are recorded with a timestamp and your name.

A.7 Response Letters

CivicRecords AI can draft a response letter for each request.

1. Open a request in **Drafting** status. 2. Click **Generate Draft Letter**. 3. The system creates a draft using the request details, the documents being released, and any exemptions confirmed. 4. Edit the draft as needed — the AI output is a starting point, not a final document. 5. Click **Save Letter** to attach it to the request record.

All generated letters include an **AI Content Disclosure** statement as required by the Colorado AI Act (CAIA) and similar state regulations. Do not remove this disclosure.

A.8 Fees

If your jurisdiction charges fees for records requests:

1. Open the request → click the **Fees** tab. 2. Enter fee line items (search time, copying, etc.). 3. Record payment when received. 4. The system tracks outstanding fees and can include a fee statement in the response letter.

A.9 Common Questions

Q: I can't find a document I know exists. The document may not be connected to the system yet. Check with your administrator which systems are currently synced. If the system is connected but a document is missing, try a different search phrase — the AI search engine matches on meaning, not just exact words.

Q: A flag was triggered on a document that clearly isn't sensitive. Click **Dismiss** on that flag. The system learns nothing from your dismissal — it only records your decision in the audit log.

Q: I advanced a request to the wrong status. Contact your administrator. Status changes are logged and can be manually corrected with an admin account. There is no automatic "back" button — the audit trail must remain intact.

Q: The deadline passed and the request is still open. Overdue requests appear with a red deadline indicator. You can still process them normally. Document the delay in the request timeline.

Q: I see a "Circuit Open" or "Paused" badge on a data source. This means the system lost connection to that data source after repeated failures. Contact your IT administrator. You can still search documents that were previously synced — only new documents from that source will be missing.

A.10 Glossary

| Term | Plain English meaning | |---|---| | **Open records request** | A formal request from a member of the public for government documents. Also called FOIA, CORA, or your state's equivalent. | | **Exemption** | A legal reason why certain information does not have to be released. Examples: Social Security numbers, active criminal investigation details, attorney-client communications. | | **PII** | Personal Identifiable Information — data that could identify a

specific person, like a Social Security number or home address. | | **Responsive document** | A document that directly answers what the requester asked for. | | **Data source** | A connected system that CivicRecords AI can search — a file folder, database, or web API. | | **Ingestion / Sync** | The process of reading documents from a data source and making them searchable. | | **Redaction** | Blacking out exempt information before releasing a document. CivicRecords AI flags candidates; staff perform the actual redaction. | | **Audit log** | A permanent, tamper-evident record of every action taken in the system. Required for legal compliance. | | **Circuit breaker** | A safety feature that pauses syncing from a data source if it fails repeatedly, preventing runaway errors. | | **AI Summary** | A short paragraph generated by the AI describing what a document contains. Used to quickly assess relevance — not a legal summary. |

Section B — Technical Reference

For IT administrators, system integrators, and power users who install, configure, and maintain CivicRecords AI.

B.1 System Requirements

| Component | Minimum | Recommended | |---|---|---| | CPU | 8 cores | 16 cores | | RAM | 32 GB | 64 GB | | Disk | 50 GB free | 2+ TB NVMe | | OS | Windows 10/11, macOS 13+, Ubuntu 22.04+, Debian 12+ | Ubuntu 22.04 LTS | | Runtime | Docker Desktop (Windows/macOS) or Docker Engine (Linux) | Docker Engine 24+ |

GPU (optional but recommended):

- NVIDIA (CUDA) — Windows and Linux
- AMD (ROCm) — Linux only
- AMD/Intel (DirectML) — Windows
- CPU fallback is supported but significantly slower for LLM inference

---

B.2 Installation

There are two supported install paths. Pick the one that matches your platform and how you prefer to install software.

> **Install paths currently shipped:** > > 1. **Windows double-click installer (T5E, UNSIGNED).** A real `.exe` installer built with Inno Setup 6.x is produced on every `v*` tag and published to GitHub Releases as `CivicRecordsAI-<version>-Setup.exe`. **It is unsigned by design for this release** (Scott-locked B3=α posture) — Windows SmartScreen will show "Windows protected your PC — Unknown publisher." on first run. Click **More info** → **Run anyway** to proceed. A SHA-256 checksum is published alongside each release asset for independent verification. The installer bundles the repo snapshot, runs a prerequisite check (Docker Desktop, WSL 2 + Virtual Machine Platform, 32 GB RAM floor, optional host Ollama), then runs `install.ps1` through `launch-install.ps1`. See `installer/windows/README.md` for the full SmartScreen walkthrough, the split Start/Install shortcut model, and checksum-verify steps. > 2. **Script-based install (macOS / Linux — and Windows if you prefer CLI).** The scripts below configure and launch the Docker Compose stack. They do **not** install Docker Desktop, Docker Engine, WSL, or any other system prerequisite — those must be present before the scripts run. If Docker is not installed, the scripts fail with a clear error and you must install Docker manually before retrying. > > **Cross-platform parity:** No native installer ships for macOS or Linux. That parity is explicit follow-on work and is not scheduled. macOS and Linux operators use the script path below.

Before you begin — prerequisites:

1. Install **Docker Desktop** (Windows 10/11 or macOS 13+): docker.com/get-started *Linux:* Install **Docker Engine** 24+ and Docker Compose v2. 2. Ensure Docker is running (you should see the Docker icon in your taskbar/menu bar, or `docker info` returns without error). 3. Confirm system requirements: 8+ CPU cores, 32 GB RAM, 50 GB free disk.

Windows (double-click installer): Download `CivicRecordsAI-<version>-Setup.exe` from the GitHub Releases page for the tag you want, double-click it, acknowledge the SmartScreen "Unknown publisher" prompt (expected — see above), and follow the installer prompts. On first launch the installer automatically runs the full bootstrap (prereq check → model pull → first-boot seed).

Windows (script path): ``` `powershell git clone https://github.com/CivicSuite/civicrecords-ai.git cd civicrecords-ai .\install.ps1 `

macOS / Linux: `bash git clone https://github.com/CivicSuite/civicrecords-ai.git cd civicrecords-ai bash install.sh `

The installer: 1. Creates .env from .env.example (prompts for required values; generates a strong admin password when run interactively) 2. Pulls Docker images

(~8–12 GB first time) 3. Pulls nomic-embed-text and the Gemma 4 tag you select in the 4-model picker (default gemma4:e4b) 4. Starts all 7 Docker Compose services 5. Runs database migrations 6. Creates the initial admin account 7. **First-boot seed (T5B):** on the very first boot, app/main.lifespan auto-seeds 175 state-scoped exemption rules across 51 jurisdictions, 5 compliance templates, and 12 notification templates. The seed is idempotent — a subsequent boot reports created=0, skipped=175/5/12 and preserves any admin customizations.

After installation, open http://localhost:8080 in your browser.

B.3 Environment Variables

All configuration lives in .env in the repo root. Never commit this file.

Variable	Required	Default	Description
DATABASE_URL	Yes	—	PostgreSQL connection string (asyncpg format)
JWT_SECRET_FILE	Yes	/run/secrets/jwt_secret	Mounted secret file for JWT signing
FIRST_ADMIN_EMAIL	Yes	—	Initial admin account email
FIRST_ADMIN_PASSWORD_FILE	Yes	/run/secrets/first_admin_password	Mounted secret file for the initial admin account password
CIVICRECORDS_SECRET_DIR	Yes	./data/secrets	Host directory used by Docker Compose for file-backed secrets
OLLAMA_BASE_URL	No	http://ollama:11434	Ollama API endpoint
REDIS_URL	No	redis://redis:6379/0	Redis connection string
SMTP_HOST	No	—	SMTP server for email notifications
SMTP_PORT	No	587	SMTP port
SMTP_USERNAME	No	—	SMTP auth username
SMTP_PASSWORD	No	—	SMTP auth password
AUDIT_RETENTION_DAYS	No	1095	Audit log retention (3 years default)
PORTAL_MODE	No	private	private (staff-only, default) or public (adds the minimal resident surface described in B.3.2)
ENCRYPTION_KEY	Yes	—	Fernet key used to encrypt data_sources.connection_config at rest. Installer auto-generates on fresh installs; **back it up separately from the database** (see B.3.3)

B.3.1 Secrets Handling

CivicRecords AI v1.6.0 stores the JWT signing secret and first-admin password in files instead of container environment variables. On Linux and macOS, install.sh writes ./data/secrets/jwt_secret and ./data/secrets/first_admin_password with 0400 permissions. Docker Compose mounts those files at /run/secrets/jwt_secret and /run/secrets/first_admin_password ; .env contains only the *_FILE pointers.

To verify the secret material is not exposed in a running API container:

```bash
docker exec <records-api-container> env | grep -E "JWT_SECRET|FIRST_ADMIN_PASSWORD"
```

The command should return no lines.

Upgrade from v1.5.x: if your .env still contains JWT_SECRET= or FIRST_ADMIN_PASSWORD= , re-run install.sh or install.ps1 . The runtime no longer treats those env vars as the Docker deployment path because any operator with container access can recover them with docker exec env .

Secrets handling on Windows

install.ps1 writes the same two secret files under .\data\secrets\ and uses icacls to remove inherited ACLs, granting read access only to SYSTEM, Administrators, and the current Windows account that runs Docker Desktop. This is the Windows equivalent of the Linux/macOS 0400 file mode for local Docker Desktop deployments.

B.3.2 Portal Mode (Private vs. Public)

CivicRecords AI can run in one of two modes. You pick the mode when you install, and you can change it later by editing .env and restarting the stack. **If you do nothing, the system runs in private mode** — the same behavior CivicRecords AI has always had.

What each mode does

Private mode (default). Only city staff can use the system. Someone visiting the site from the open internet sees the staff login screen and nothing else. There is no "register" button. There is no public page. Residents who want to file a request still do so the way they always have — by email, phone, or walking into city hall. Staff receive those requests on their side as they do today.

Choose private mode if:

- Your city does not yet have policies, staffing, or legal sign-off to accept resident-submitted requests through a website.
- You want to deploy the staff tool first and add the public surface later.
- You are deploying into an internal network and public access is not desired.

Public mode. Adds three — and only three — resident-facing pages on top of the staff tool: 1. A **public landing page** that explains what residents can do online. 2. A **resident registration page** where a member of the public can create an account for themselves. 3. A **request submission form** that a registered, signed-in resident can use to file a records request.

A resident must create an account and sign in before they can submit a request. There is no anonymous walk-up submission through the website — the person submitting the request has to be a real, identifiable account holder. This is a deliberate design decision for the first public-mode slice.

What public mode does NOT add. There is no public search of already-released records, no full resident dashboard, no "track my request" screen. Those features are on the roadmap but are not part of this release. If a resident asks, the honest answer is "not yet — for now, your submission confirmation and any follow-up go through email."

Choosing the mode at install time

The installer will ask you which mode to run in.

On Windows (running install.ps1 or the Start-Menu "Install or Repair CivicRecords AI" shortcut):

```
Portal mode — private (staff-only) or public (adds minimal resident surface)?
Press Enter for the default [private], or type "public":
```

On Linux / macOS (running bash install.sh):

```
Portal mode [private]:
```

Press Enter to accept the default (private), or type public and press Enter.

Non-interactive installs (for example, when you are scripting a deployment): set the environment variable CIVICRECORDS_PORTAL_MODE=public before running the installer. The installer will pick it up and skip the prompt.

Changing the mode after installation

You can switch modes at any time without reinstalling. You will need access to the server where CivicRecords AI is running and permission to restart the Docker stack.

1. Open .env in the CivicRecords AI install folder (the same folder that has docker-compose.yml). 2. Find the line that reads PORTAL_MODE=private (or add it if it is missing). 3. Change it to PORTAL_MODE=public — or the reverse, if you are switching back to private. 4. Save the file. 5. Restart the stack: docker compose restart api frontend (from that folder). 6. Open http://localhost:8080/ in a browser and confirm you see the expected screen — the staff login in private mode, or the public landing page in public mode.

If the site does not come up the way you expect, the most common cause is a typo in .env. The system only accepts exactly private or public (case and extra spaces are forgiven — Public and PUBLIC both work — but misspellings like publick will stop the system from starting. If the stack refuses to start, open the logs with docker compose logs api and look for a message about PORTAL_MODE; fix the typo and restart.

Who can do what, at a glance

| Who | Private mode | Public mode | |---|---|---| | Staff (admin, staff, reviewer, read-only, liaison) | Sign in at the staff login → staff workbench | Sign in at the staff login → staff workbench (unchanged) | | Someone who is not signed in | Sees staff login only | Sees public landing, can register, can sign in | | A registered resident (signed in) | N/A — resident accounts cannot be created | Sees the three public pages: landing, their registration state, and the submission form | | Anonymous visitor trying to submit a request | Not possible — no public surface | Not possible — submission requires a signed-in resident account |

Glossary

- **.env** — A plain-text configuration file in the CivicRecords AI install folder that holds settings like database passwords, mail server settings, and now PORTAL_MODE. Never share this file with anyone outside your IT team.
- **Docker stack** — The collection of running services (database, API, frontend, etc.) that make up CivicRecords AI.
- **Resident account** — A user account with the "public" role. Can submit requests but cannot view other people's requests or use any staff tools.

B.3.3 Encryption Key for Connector Credentials (ENG-001 / Tier 6)

CivicRecords AI encrypts the credentials you enter for each connected data source — API keys, bearer tokens, OAuth2 client secrets, Basic-auth passwords, and

database connection strings — before writing them to PostgreSQL. The encryption is driven by a single environment variable, `ENCRYPTION_KEY`.

What the key protects

Without the key, connector credentials would sit in the `data_sources.connection_config` column as plaintext JSON — visible to any PostgreSQL superuser, to `pg_dump` output, and to anyone with a restored backup file. With the key, the column stores a Fernet envelope (`{"v": 1, "ct": "..."}`) that is opaque without the key. A stolen backup by itself is not enough to read the credentials; an attacker would need both the backup **and** the key.

This protects: API keys, OAuth2 client secrets, Basic-auth passwords, database connection strings, and any other sensitive field an admin typed into the Add Source wizard. It does **not** protect the raw ingested documents, search indexes, or request content — those are separate surfaces.

Back up the key separately from the database — this is critical

Losing `ENCRYPTION_KEY` means losing every saved connector configuration. There is no recovery path. If the key is lost, every row in `data_sources.connection_config` becomes unreadable ciphertext and every connector must be re-entered by hand.

Do not store the key in the same place as the database backup. If your database backup tape or cloud bucket also contains the `.env` file with the key, the two protections collapse into one and a single breach leaks both. Put the key somewhere physically or organizationally separate — a password manager under IT lock, a sealed envelope in a safe, a separate credential vault — and document who knows where it is.

Generating a key manually

The installer generates a key automatically on a fresh install and writes it to .env with a loud red "BACK THIS UP SEPARATELY" banner. If you need to generate one by hand — for example, setting up .env in a new environment or filling in an .env.example placeholder — run:

```bash
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Copy the output into .env as ENCRYPTION_KEY=... . The value must be a real URL-safe-base64 Fernet key; the system rejects common insecure defaults and obviously-malformed keys at startup rather than starting in a broken state.

Verifying encryption after install or upgrade

Once the stack is running and the Tier 6 migration has applied, confirm every existing row is encrypted:

```bash
docker compose run --rm --no-deps api python scripts/verify_at_rest.py
```

Exit code 0 means every data_sources.connection_config row is envelope-shaped (encrypted). Exit code 1 means at least one row is still plaintext — stop and investigate before continuing. The admin UI behavior is unchanged in both cases: connector configs decrypt transparently on read, so a working admin GET does **not** by itself prove the column is encrypted.

Key rotation is not supported in this release

This release supports a single active key. There is no procedure for rotating ENCRYPTION_KEY without manually re-encrypting every row. The envelope format carries a "v": 1 version tag so a future release can add rotation, but for now:

pick a key at install time, back it up, and keep using it. If you believe the key has been compromised, contact the project maintainers — rotation tooling is tracked as a follow-on slice.

What if I change or lose the key?

- **Changed to a new key:** existing rows become unreadable. The admin UI will return errors when opening a saved source. Restore the original key from backup to recover.
- **Lost entirely (no backup):** the saved configurations cannot be recovered. Delete the affected rows and re-enter each connector from the admin UI. This is irreversible — which is why the backup instruction above is emphasized.

B.4 Docker Services

Seven services run via Docker Compose:

Service	Image	Port	Role
postgres	postgres:17 + pgvector	5432	Primary database + vector store
redis	redis:7.2	6379	Celery broker and result backend
ollama	ollama/ollama	11434	Local LLM inference
api	civicrecords-ai/api	8000	FastAPI backend
worker	civicrecords-ai/api	—	Celery async task worker
beat	civicrecords-ai/api	—	Celery beat scheduler
frontend	civicrecords-ai/frontend	8080	nginx + React SPA

Common operations: `bash

Start all services

```
docker compose up -d
```

Stop all services

```
docker compose down
```

View logs

```
docker compose logs -f api docker compose logs -f worker
```

Run database migrations manually

```
docker compose exec api alembic upgrade head
```

Restart a single service

```
docker compose restart worker `
```

B.5 Connector Types and Configuration

`CivicRecords AI uses a standardized connector framework. Each connector must implement:` authenticate() , discover() , fetch() , and health_check() .

B.5.0 Adding a Data Source (Wizard Walkthrough)

Admins add data sources through the **Sources** → **Add Source** button. The dialog is a three-step wizard:

Step 1 — Source name and type. 1. Enter a **Source Name** (required). This is how the source appears in listings and sync logs. Pick something specific — "City Clerk Email Archive" is better than "Source 1". If you skip this and click **Next**, the wizard stays on Step 1 and shows a red message under the input: *"Enter a name for this source — this is how you will identify it later."* 2. Pick a **Source Type** from the four choices: **File System** (local or mounted folder), **Manual Drop** (watched drop folder), **REST API** (HTTP endpoint), or **ODBC / Database** (SQL database via ODBC). The selection is keyboard-navigable — Tab into the group, then arrow keys to choose.

Step 2 — Connection details (the fields depend on the type you picked).

- **File System / Manual Drop:** enter the **Directory Path** (required). This must be a path visible to the Docker container (for example `/mnt/records` on Linux or a mounted Windows share). If you leave it blank, the wizard shows: *"Enter the full directory path where documents live (for example, /mnt/records)."*

- **REST API:** enter the **Base URL** (required, must start with `http://` or `https://`) and optional **Endpoint Path**, then pick an **Authentication** method (None / API Key / Bearer Token / OAuth 2.0 / Basic Auth) and fill in the credentials it reveals. Each authentication method has its own required fields — leaving any of them blank produces a specific message naming the field (for example *"Client secret is required for OAuth 2.0."*).

- **ODBC / Database:** enter the **Connection String** (required, stored encrypted), **Table Name** (required), **Primary Key Column** (required — used to track which rows have been ingested), and optional **Modified Timestamp Column** and **Batch Size**.

  Step 3 — Review, schedule, and create. 1. Check **Enable automatic sync** to let the source sync on a schedule (on by default). Pick a preset from the dropdown (every 15 minutes, hourly, nightly at 2 AM UTC, weekly, etc.) or choose **Custom…** to enter a 5-field cron expression directly. Invalid cron expressions produce a message: *"Sync schedule must be a valid 5-field cron expression (for example, 0 2)."* 2. Review the summary card — it shows the name, type, and key configuration values (secrets are masked). 3. Click **Test Connection** to verify the credentials and destination. A green checkmark means the source is reachable; a red X shows the specific error returned by the connector. 4. Click **Create Source** to save. If the backend rejects the create (invalid field, duplicate name, credential verification failure), the error is shown in a red banner at the bottom of the dialog with the exact message and the form is preserved so you can correct and retry.

  What to do when a field is rejected. Every validation message names the field and gives you a concrete example. The field with the problem is outlined and will announce as invalid to screen readers. Typing into the field clears that field's error immediately — you don't have to click Next again to clear the red.

B.5.1 File System Connector

Reads files from a local or network-mounted directory.

```
`json { "source_type": "file_system", "connection_config": { "path": "/mnt/city-documents" } } `
```

| Field | Description | |---|---| | path | Absolute path visible to the Docker container (local dir or network mount) |

Supported file types: PDF, DOCX, XLSX, CSV, TXT, HTML, EML. Scanned PDFs processed via Gemma 4 multimodal + Tesseract OCR fallback.

B.5.2 Manual Drop Connector

Watches a drop folder for files uploaded manually by staff. Use when a system has no API and staff exports files by hand.

```
json { "source_type": "manual_drop", "connection_config": { "drop_path": "/mnt/drop/incoming" } }
```

| Field | Description | |---|---| | drop_path | Directory the connector watches for new files |

Workflow: Staff places exported files in the drop folder. On each sync, the connector ingests any files not yet processed, then leaves them in place (files are never deleted).

B.5.3 REST API Connector

Connects to any REST API that returns JSON, XML, or CSV.

```
json { "source_type": "rest_api", "connection_config": { "base_url": "https://api.tyler-munis.example.gov", "endpoint_path": "/v1/contracts", "auth_method": "api_key", "key_header": "X-API-Key", "key_location": "header", "api_key": "your-api-key-here", "pagination_style": "page", "page_param": "page", "limit_param": "per_page", "page_size": 100, "results_field": "data", "id_field": "id", "response_format": "json", "max_records": 50000, "max_response_bytes": 10485760 } }
```

Auth methods: none, api_key (header or query), bearer, oauth2 (client credentials), basic

Pagination styles: `none`, `page`, `offset`, `cursor`

OAuth2 configuration: `` `json { "auth_method": "oauth2", "token_url": "https://auth.example.gov/token", "client_id": "your-client-id", "client_secret": "your-client-secret" } ` ``

Rate limiting: The connector honors `Retry-After` response headers, sleeping up to 600 seconds before retrying (D10 spec). Malformed headers fall back to exponential backoff.

B.5.4 ODBC Connector

Connects to SQL databases via pyodbc (SQL Server, MySQL, PostgreSQL, Oracle, SQLite).

`` `json { "source_type": "odbc", "connection_config": { "connection_string": "DRIVER={ODBC Driver 17 for SQL Server};Server=10.0.1.5;Database=TylerNewWorld;UID=readonly_user;PWD=password", "table_name": "documents", "pk_column": "id", "modified_column": "updated_at", "batch_size": 500 } } ` ``

Field	Description
connection_string	Full ODBC connection string (never logged or echoed)
table_name	Table to ingest. Each row becomes one document.
pk_column	Primary key column for deduplication
modified_column	Optional timestamp column — used for incremental sync
batch_size	Rows fetched per query (default 500)

Security: Only `SELECT` queries are issued. `table_name` and `pk_column` are validated against a safe-identifier pattern — SQL injection via those fields is blocked at schema validation time.

B.6 Scheduled Sync

Each data source can be synced on a schedule using standard 5-field cron expressions.

```
` ┌──────────── minute (0–59) │ ┌──────────── hour (0–23) │ │ ┌──────────── day of month (1–31) │ │ │ ┌──────────── month (1–12) │ │ │ │ ┌──────────── day of week (0–6, Sunday=0) │ │ │ │ │ 0 2 * # Every day at 2:00 AM UTC */30 * # Every 30 minutes 0 */6 # Every 6 hours `
```

Constraints:

- Minimum interval: 5 minutes (enforced by validation)
- Minimum re-run interval: 7 days rolling window (prevents accidentally-tight schedules)
- All schedules are evaluated in UTC; the UI shows your local timezone for reference
- `Set` schedule_enabled = false `to pause scheduling without clearing the schedule expression`

B.7 Sync Failure Tracking and Circuit Breaker

Failure States

Individual records track their own failure state:

`| Status | Meaning | |---|---| |` retrying `| Failed at least once; will retry on next sync | |` permanently_failed `| Failed ≥ 5 times or is ≥ 7 days old; no longer retried automatically | |` resolved `| Was failing; successfully ingested on a later sync | |` tombstone `| Admin-dismissed; excluded from future syncs |`

Circuit Breaker

The circuit breaker tracks *full-run* failures — when every single record in a sync run fails.

- After **5 consecutive full-run failures**: source is auto-paused (sync_paused = true), admin is notified, health status shows **Circuit Open**
- To recover: fix the underlying issue → click **Unpause** → on the source card
- After unpausing: the source enters **grace period** (threshold = 2 consecutive full-run failures to re-trip). This gives fast feedback if the fix didn't work.
- A successful sync after unpausing resets the threshold to normal (5)

Health Status

Each source card shows a colored health indicator:

| Color | Status | Meaning | |---|---|---| | 🟢 Green | Healthy | Syncing normally | | 🟡 Amber | Degraded | Some record failures, but sync is running | | 🔴 Red | Paused | Circuit open; manual intervention required |

Failed Records Panel

Click **View failures** on any source card with failures to open the panel:

- See each failed record's path, error message, retry count, and status
- **Retry** individual records or use **Retry all permanently failed** for bulk retry
- **Dismiss** records you don't want ingested (adds a tombstone; preserved in audit log)

B.8 API Reference

The FastAPI backend exposes a REST API documented at http://localhost:8000/docs (Swagger UI) and http://localhost:8000/redoc .

Authentication: All endpoints require a JWT bearer token.

`bash

Obtain a token

```
curl -X POST http://localhost:8000/auth/login \ -H "Content-Type: application/json" \ -d '{"email": "admin@example.gov", "password": "yourpassword"}'
```

Use the token

```
curl http://localhost:8000/datasources \ -H "Authorization: Bearer <token>" `
```

Key endpoint groups:

```
| Prefix | Description | |---|---| | /auth/ | Login, token refresh, password management | | /datasources/ | CRUD for data sources, sync triggers, failure management | | /documents/ | Document search, retrieval, export | | /requests/ | Records request lifecycle | | /exemptions/ | Flag review and management | | /admin/ | Users, departments, model registry | | /compliance/ | Compliance templates download | | /health | Service health check |
```

Service accounts: Create dedicated API accounts with limited roles for programmatic access (e.g., a requester portal integration). Manage in Admin → Service Accounts.

B.9 Model Registry

CivicRecords AI uses two model types:

```
| Type | Default | Notes | |---|---|---| | **Embedding** | nomic-embed-text | Always required; very lightweight | | **Chat/Vision** | gemma4:e4b (default) | Used for document analysis, AI summaries, and OCR on scanned PDFs |
```

Supported Gemma 4 models (the installer picker presents all four; only e2b and e4b are supportable at the 32 GB baseline target profile, 26b and 31b require stronger hardware):

| Tag | Class | Disk | RAM (advisory) | Supportable at baseline | |---|---|---|---|---| | gemma4:e2b | Edge 2.3B effective | 7.2 GB | ~16 GB | yes | | gemma4:e4b | Edge 4.5B effective (DEFAULT) | 9.6 GB | ~20 GB | yes | | gemma4:26b | Workstation MoE 25.2B/3.8B active | 18 GB | 48+ GB recommended | no | | gemma4:31b | Workstation dense 30.7B | 20 GB | 64+ GB, GPU recommended | no |

To change models: 1. Go to **Admin → Model Registry** 2. Select the model type 3. Enter the Ollama model identifier (e.g., gemma4:e2b, mistral:7b, llama3:8b) 4. Click **Save** — the change takes effect on the next task that requires that model

To pull a new model: `bash docker compose exec ollama ollama pull gemma4:e4b`

B.10 Backup and Maintenance

Database backup: `bash docker compose exec postgres pg_dump -U civicrecords civicrecords | gzip > backup_$(date +%Y%m%d).sql.gz`

Restore: `bash gunzip -c backup_20260101.sql.gz | docker compose exec -T postgres psql -U civicrecords civicrecords`

Disk space: Document content and embeddings grow with your document set. Monitor with: `bash docker compose exec postgres psql -U civicrecords -c "SELECT pg_size_pretty(pg_database_size('civicrecords'));"`

Audit log retention: Set AUDIT_RETENTION_DAYS in .env. Default is 1095 (3 years). A Celery beat task trims old records nightly.

Data sovereignty verification: `bash

Linux/macOS

`bash scripts/verify-sovereignty.sh`

Windows

`.\scripts\verify-sovereignty.ps1` ` Runs netstat and confirms no outbound connections are active. Present output to your network security team.

B.11 Troubleshooting

Services won't start: `bash docker compose logs postgres # Check DB initialization docker compose logs ollama # Check model download docker compose ps # Check which services are unhealthy `

API returns 500: `bash docker compose logs api --tail=50 `

Worker not processing tasks: `bash docker compose logs worker --tail=50 docker compose restart worker `

Ollama model not found: `bash docker compose exec ollama ollama list # See installed models docker compose exec ollama ollama pull gemma4:e4b # Pull the default if missing `

Frontend shows blank page: Check browser console for errors. Most common cause: API is unreachable. Verify docker compose ps shows api as healthy.

Database migration failed: `bash docker compose exec api alembic current # See current revision docker compose exec api alembic history # See migration history

```
docker compose exec api alembic upgrade head # Rerun migrations `
```

Section C — Architectural Reference

For developers and architects who need to understand the system's structure, data flow, and design decisions.

C.1 System Overview

CivicRecords AI runs as seven Docker Compose services communicating over an internal Docker network. All data stays on the host machine — no external dependencies after initial setup.

```
`mermaid graph TB subgraph Browser["Browser (React 18 + shadcn/ui)"] UI[Admin SPA] end

subgraph Frontend["nginx :8080"] SPA[Static Assets] end

subgraph API["FastAPI :8000"] Auth[Auth / JWT] Routes[Routers] Search[Search Engine] Pipeline[Ingestion Pipeline] end

subgraph Workers["Celery"] Worker[Task Worker] Beat[Beat Scheduler] end

subgraph Data["Data Layer"] PG[(PostgreSQL 17\n+ pgvector)] Redis[(Redis 7.2)] end

subgraph AI["AI Layer"] Ollama[Ollama LLM/Embed] end

subgraph Sources["City Data Sources"] FS[File System] REST[REST APIs] DB[SQL Databases\nvia ODBC] end

UI --> Frontend Frontend --> API API --> PG API --> Redis API --> Ollama Worker --> PG Worker --> Redis Worker --> Ollama Worker --> Sources Beat --> Redis `
```

Design principles:

- **Local-first:** All inference, storage, and processing on-premises. No cloud calls.
- **Human-in-the-loop:** No automatic releases, redactions, or status advances.
- **Audit by default:** Every state change writes an immutable audit log entry.
- **Idempotent ingestion:** Re-syncing the same document is safe; deduplication prevents duplicates.

C.2 Data Model

```
```mermaid classDiagram class DataSource { +UUID id +String name +SourceType source_type +JSON connection_config +bool is_active +bool sync_paused +String sync_paused_reason +int consecutive_failure_count +String health_status +UUID department_id +authenticate() +discover() +fetch() +health_check() }
```

class Document { +UUID id +String source_path +String content_hash +String file_type +int file_size +String parsed_text +UUID source_id }

class Chunk { +UUID id +UUID document_id +int chunk_index +String text +Vector embedding +int token_count }

class SyncFailure { +UUID id +UUID source_id +String source_path +String error_message +String error_class +int retry_count +String status +DateTime first_failed_at }

class SyncRunLog { +UUID id +UUID source_id +DateTime started_at +DateTime finished_at +String status +int records_attempted +int records_succeeded +int records_failed }

class Request { +UUID id +String reference_number +String requester_name +String description +RequestStatus status +DateTime legal_deadline +UUID department_id +UUID assigned_to }

class ExemptionFlag { +UUID id +UUID document_id +UUID request_id +String flag_type +String matched_text +String rule_name +String decision +UUID decided_by }

class User { +UUID id +String email +UserRole role +UUID department_id +bool is_active }

class Department { +UUID id +String name +String code }

class AuditLog { +UUID id +String action +UUID actor_id +JSON payload +String previous_hash +String entry_hash }
```

```
DataSource "1" --> "many" Document DataSource "1" --> "many" SyncFailure DataSource "1" --> "many" SyncRunLog Document "1" --> "many" Chunk Document "1" --> "many" ExemptionFlag Request "1" --> "many" ExemptionFlag User "many" --> "1" Department DataSource "many" --> "1" Department Request "many" --> "1" Department
```

Key design decisions:

- **Content hash deduplication:** Documents are keyed by `source_path` for structured connectors (REST API, ODBC) and by `content_hash` for binary files. Re-syncing an unchanged document is a no-op.
- **Chunk + Embedding:** Each document is split into ~512-token chunks. Each chunk has its own pgvector embedding. Search queries both the vector index and PostgreSQL full-text index, then merges with reciprocal rank fusion.
- **Hash-chained audit log:** Each `AuditLog` row includes the SHA-256 hash of the previous row. Tamper detection is O(n) chain verification.

C.3 Ingestion Pipeline

```
mermaid sequenceDiagram participant Beat as Celery Beat participant Worker as Celery Worker participant Connector as Connector participant Pipeline as Ingestion Pipeline participant DB as PostgreSQL
```

```
Beat->>Worker: trigger_sync(source_id) [cron] Worker->>DB: SELECT data_source WHERE id=source_id Worker->>Connector: authenticate() Worker->>DB: SELECT retrying SyncFailures (Layer 2 retry) loop For each retrying failure Worker->>Connector: fetch(source_path) alt Success Worker->>Pipeline: ingest(content) Pipeline->>DB: UPDATE SyncFailure status=resolved else Failure Worker->>DB: UPDATE SyncFailure retry_count++ end end Worker->>Connector: discover(since=last_cursor) loop For each discovered record Worker->>Connector: fetch(source_path) alt Success Worker-
```

>>Pipeline: parse → chunk → embed → upsert Pipeline->>DB: INSERT/UPDATE Document, Chunk, Embedding else Failure Worker->>DB: INSERT SyncFailure (status=retrying) end end Worker->>DB: UPDATE DataSource last_sync_at, cursor alt All records failed Worker->>DB: consecutive_failure_count++ alt Count >= threshold Worker->>DB: sync_paused=true (circuit open) end else Any record succeeded Worker->>DB: consecutive_failure_count=0 end Worker->>DB: INSERT SyncRunLog `

Two-layer retry:

- **Layer 1 (task-level):** The Celery task uses exponential backoff with jitter for transient connection errors (handled by with_retry() in retry.py).
- **Layer 2 (record-level):** Individual record failures are persisted as SyncFailure rows and retried on subsequent sync ticks, up to 5 retries over 7 days.

Idempotency contract:

- Structured records (REST API, ODBC): keyed by source_path . Same path → UPDATE in-place, chunks and embeddings replaced atomically.
- Binary records (filesystem): keyed by content_hash . Same hash → no-op.
- Race conditions: SELECT FOR UPDATE on the document row + partial unique indexes prevent duplicate inserts under concurrent workers.

C.4 Records Request Lifecycle

`mermaid sequenceDiagram participant Staff as Staff User participant API as FastAPI participant DB as PostgreSQL participant AI as Ollama

Staff->>API: POST /requests (requester info, description) API->>DB: INSERT Request (status=intake) API->>DB: INSERT AuditLog (action=request_created)

```
Staff->>API: PUT /requests/{id}/status (assigned) API->>DB: UPDATE Request status=assigned API->>DB: INSERT AuditLog

Staff->>API: POST /requests/{id}/search API->>AI: embed(description) API->>DB: pgvector similarity search + FTS merge API->>DB: INSERT SearchLog (query, result_count, document_ids) API-->>Staff: ranked document list

Staff->>API: POST /requests/{id}/documents (mark responsive) API->>DB: INSERT RequestDocument (responsive=true)

Staff->>API: POST /requests/{id}/exemptions/analyze API->>DB: Run PII patterns + statutory keyword rules API->>AI: (optional) LLM secondary review API->>DB: INSERT ExemptionFlag[] (decision=pending)

Staff->>API: PUT /exemptions/{flag_id} (confirm/dismiss) API->>DB: UPDATE ExemptionFlag decision=confirmed/dismissed API->>DB: INSERT AuditLog (decision, actor, timestamp)

Staff->>API: POST /requests/{id}/letter/generate API->>AI: draft_letter(request_context, exemptions) API-->>Staff: draft letter text
```

```
Staff->>API: PUT /requests/{id}/status (fulfillment) API->>DB: UPDATE Request status=fulfillment API->>DB: INSERT AuditLog `
```

Human-in-the-loop enforcement: The API refuses status transitions that skip required steps. A request cannot move from `intake` directly to `fulfillment` — every intermediate status must be visited. This is enforced at the API layer, not just the UI.

C.5 Sync Failure and Circuit Breaker State Machine

```
`mermaid stateDiagram-v2 [*] --> Healthy : source created / sync successful
```

```
Healthy --> Degraded : fetch failure logged\n(SyncFailure: retrying) Degraded --> Healthy : subsequent sync succeeds\nconsecutive_failure_count reset to 0
```

```
Degraded --> Degraded : retry attempt fails\n(retry_count < 5 AND age < 7 days)
Degraded --> PermanentlyFailed : IntegrityError OR\nretry_count ≥ 5 OR age ≥ 7 days

Healthy --> CircuitOpen : 5 consecutive FULL-RUN failures\n(sync_paused = true)
Degraded --> CircuitOpen : 5 consecutive FULL-RUN failures\n(sync_paused = true)

CircuitOpen --> Healthy : admin unpauses source\nAND next sync succeeds\n(grace threshold = 2) CircuitOpen --> CircuitOpen : admin unpauses\nbut sync still fails\n(grace threshold exhausted → re-paused)
```

```
PermanentlyFailed --> Dismissed : admin dismisses record\n(soft delete, audit trail preserved) PermanentlyFailed --> Healthy : admin clicks Retry\nAND next sync succeeds `
```

```
**Grace period implementation:** When an admin clicks **Unpause** →, the database column sync_paused_reason is set to "grace_period". The sync runner reads this value and uses threshold=2 instead of threshold=5 for the next sync run. On successful sync, the sentinel is cleared. This provides fast feedback: if the underlying problem wasn't actually fixed, the circuit trips again after just 2 failures.
```

C.6 Deployment Topology

 Deployment stack — entire system runs inside Docker Compose on the city's network. No cloud, no outbound by default.

```
`mermaid graph TB subgraph Host["City Server (on-premises)"] subgraph Docker["Docker Compose Network"] FE["frontend\nnginx :8080"] API["api\nFastAPI :8000"] W["worker\nCelery"] B["beat\nCelery Beat"] PG["postgres\nPostgreSQL 17\n+ pgvector :5432"] R["redis\nRedis 7.2\n:6379"] OL["ollama\nOllama :11434"] end subgraph Storage["Host Volumes"] PGVol["postgres_data"] OLVol["ollama_models"] DocVol["document_mounts"] end end subgraph Network["City Network"] Browser["Staff Browser"] MuniSystems["Municipal Systems\n(REST APIs, Databases,\nFile Shares)"] end Browser --> FE FE --> API API --> PG API --> R API --> OL W --> PG W --> R W --> OL W --> MuniSystems B --> R PG --- PGVol OL --- OLVol W --- DocVol `
```

Network isolation: The Docker Compose network is internal. Only frontend (:8080) and api (:8000) are exposed to the host network. All inter-service communication is internal. No service initiates outbound internet connections after initial setup.

C.7 Search Architecture

 LLM call flow — records-ai application code routes through civiccore.llm (context assembly, template resolution, model registry, provider factory) to a local Ollama provider, with optional cloud providers behind opt-in extras.

Hybrid search combines two retrieval strategies and merges them with reciprocal rank fusion:

```
User Query
  │
  ├──► Semantic Search (pgvector)
  │      Embed query → cosine similarity against Chunk.embedding
  │      Returns: top-K chunks with similarity scores
  │
  └──► Keyword Search (PostgreSQL FTS)
         to_tsquery(query) → ts_rank against Document.fts_vector
         Returns: top-K documents with rank scores
  │
  ▼
Reciprocal Rank Fusion
  score = Σ 1/(k + rank_i) for each result list
  │
  ▼
Deduplicate → Normalize scores 0–1
  │
  ▼
Optional: LLM Summary Generation (Ollama)
  │
  ▼
Return: ranked DocumentResult[] with source attribution
```

Normalization: Scores are normalized to [0, 1] within each result set before fusion, so semantic and keyword results are weighted equally regardless of their native score scales.

Source attribution: Every result includes the originating DataSource, source_path, and Document.id for full traceability.

C.8 Security Architecture

 Sovereignty boundary — all runtime components (FastAPI, Celery, Postgres+pgvector, Ollama, local volumes) live inside the city's on-prem network. No outbound by default; cloud is opt-in only. Connector credentials are encrypted at rest with Fernet.

Authentication: JWT tokens with configurable expiry. Tokens are signed with JWT_SECRET using HS256. Refresh tokens are stored in Redis with revocation support.

Authorization: Role-based access control enforced at the API layer (FastAPI dependency injection):

- require_role(UserRole.ADMIN) — admin-only endpoints
- require_role(UserRole.STAFF) — staff and above
- Department scoping injected via get_current_user() — all queries are automatically filtered to the user's department

Audit log integrity: Each audit entry includes the SHA-256 hash of the previous entry (previous_hash). Full chain verification can detect any tampering. The chain can be verified with: `bash docker compose exec api python -m app.audit.verify_chain `

SQL injection prevention: ODBC connector queries are validated against an allowlist pattern before execution. No user input is ever interpolated into query strings.

No secrets in code: All credentials live in .env (gitignored). The verify-sovereignty.sh` script confirms no outbound connections.

C.9 Key Architectural Decisions

| Decision | Choice | Rationale | |---|---|---| | LLM runtime | Ollama (local) | Data sovereignty requirement; no resident data leaves the network | | Vector store | pgvector (PostgreSQL extension) | Eliminates a separate vector database service; transactions span relational + vector data atomically | | Task queue | Celery + Redis | Mature, well-understood; supports scheduled sync (beat) and async ingestion | | Frontend | React 18 + shadcn/ui | Accessible component primitives; civic design tokens; TypeScript strict mode | | Audit log | Hash-chained PostgreSQL table | Tamper-evident without external dependencies; meets CAIA and state compliance requirements | | Connector protocol | authenticate/discover/fetch/health_check | Standard interface enables adding new source types without changing ingestion pipeline | | Embedding model | nomic-embed-text | High quality, small footprint (2 GB), runs on CPU if no GPU present | | OCR strategy | Gemma 4 multimodal primary, Tesseract fallback | Handles handwritten and low-quality scans; Tesseract as CPU-safe fallback |

End of User Manual — CivicRecords AI v1.1+ · April 2026